

Feasibility of a Server-Side Neural-Network Anti-Cheat for Counter-Strike

Overview

This document evaluates how realistic it is to build a server-side anti-cheat system for Counter-Strike that uses neural networks to detect cheaters, similar in spirit to the AI-based project showcased by the YouTuber HaiX. It focuses on feasibility for a bachelor thesis: required skills and resources, time estimates, technical approach, constraints, and how original such a project would be in an academic context.^{[1][2]}

Existing Work and Inspiration

In HaiX's project, a small team created a server-side anti-cheat for CS2 over roughly three months, focusing on detecting trigger bots, auto-strafe, and bunny-hop scripts by building per-player profiles from server-side input events and flagging statistically improbable patterns. The system currently relies on hand-crafted logic, but the developers explicitly mention extending it with neural networks to improve detection of subtle patterns over time.^[^2]

Recent research confirms that machine-learning and deep-learning approaches to cheat detection in FPS games are feasible and increasingly practical. Examples include XGuardian, a server-side system that analyzes pitch and yaw time series to detect aim-assist cheats across CS2 and other FPS titles, and reports high detection accuracy with low overhead. Other work surveys vision-based and input-based approaches (using CNNs or RNNs) and highlights trade-offs between accuracy, latency, generalization, and privacy.^{[3][4][5][1]}

This background suggests that building a simplified, prototype-level server-side ML anti-cheat for a bachelor thesis is ambitious but technically realistic, as long as the scope is clearly limited to a narrow set of cheats and to proof-of-concept performance instead of production robustness.^{[1][3]}

Technical Approach (High-Level)

1. Problem Definition and Scope

A practical thesis project must narrow the scope to one or two cheat categories, for example:

- Aimbot or trigger bot detection based on server-side shot and crosshair movement patterns (e.g., time between enemy entering crosshair and shot, number of “one-tick” shots, unnatural reaction times).^{[6][2]}
- Movement cheats such as auto-strafe and scripted bunny-hop, detected through correlations between jump commands, velocity, and ground contact states.^{[2][6]}

The target game could be CS2, CS:GO (on community servers), or a simpler FPS environment that exposes sufficient server-side telemetry.

2. Data Collection Pipeline

The core of an ML-based anti-cheat is data:

- Instrument a game server (e.g., SourceMod plugin for CS:GO/CS2 or a custom server) to log relevant per-tick data: player view angles (pitch, yaw), firing events, hit events, movement commands, velocity, ground flags, and possibly game state such as enemy positions (if available server-side).^{[6][1]}
- Store logs in a structured format (e.g., JSON or CSV), grouped by match and player.
- Label segments of data as “legit” vs “cheat” using controlled experiments: record sessions where trusted testers play normally and sessions where the same testers use known cheats (trigger bots, simple aimbots, bunny-hop scripts) with clear instructions when to enable them.^{[7][6]}

Several academic projects emphasize that carefully designed data collection and labeling is the main bottleneck, sometimes more important than model architecture.^{[3][7][^1]}

3. Feature Engineering and Model Design

There are two viable directions:

- Hand-crafted features + classical ML: compute statistics such as distribution of time-to-fire after target enters crosshair, fraction of single-tick mouse presses, average and variance of view-angle velocity when aiming near opponents, or consistency of jump timing, then train a classifier such as random forest, gradient boosting, or shallow neural network.^{[1][6]}
- Sequence models or small deep nets on raw or lightly processed time series: treat view angles and input commands as multivariate sequences and train a 1D CNN or RNN (LSTM/GRU) to classify segments as cheat vs legit, similar to published RNN-based detectors for aimbots and behavior anomalies.^{[5][3]}

For a bachelor thesis, the first approach is more manageable and easier to interpret, while still allowing experimentation with simple neural networks on top of extracted features.^{[7][3]}

4. Server-Side Integration

To remain server-side and cheat-resistant, detection should run on the game server or a separate trusted backend:

- Implement a real-time module that ingests per-tick events and maintains rolling windows of features for each player.^{[8][1]}
- Periodically (e.g., every few seconds or at end of rounds), feed feature vectors into the trained model to produce cheat probability scores per player.
- Configure thresholds and aggregation rules (e.g., require several high-confidence windows before flagging) to control false positives.

Existing commercial AI-powered server-side anti-cheat products for engines like Unreal show that such architectures are practical and that model inference overhead can be kept low enough for real-time multiplayer gameplay.^[8]

5. Evaluation

The thesis would evaluate:

- Detection performance: precision, recall, F1 score on held-out labeled sessions.^{[3][1]}
- Robustness to player skill: ability to avoid flagging highly skilled legit players.
- Latency: number of rounds or seconds until a cheater is flagged under realistic cheat usage patterns.^{[2][1]}
- Practical deployability: CPU/memory cost per player and complexity of server integration.

Required Skills and Resources

Technical Skills

To execute the project, the student needs competence in:

- Programming for game servers (e.g., SourcePawn/SourceMod for CS, or scripting for another FPS server) to implement logging and plugin logic.^{[9][10]}
- Python for data processing and ML, including libraries like NumPy, pandas, and scikit-learn or PyTorch/TensorFlow.^{[7][3]}
- Basic machine learning and statistics: supervised classification, cross-validation, overfitting, and evaluation metrics.^{[3][7]}

Existing theses and papers on ML-based cheat detection routinely rely on similar toolchains and knowledge, suggesting this is aligned with typical upper-undergraduate capabilities in computer science.^{[7][3]}

Hardware and Software

The resource requirements are moderate for a prototype:

- A dedicated game server instance (local or hosted) where custom plugins can be installed.
- A reasonably powerful PC or university lab machine for data processing and model training; small models and datasets are sufficient, so a single GPU or even CPU-only training is plausible.^{[1][3]}
- Access to or creation of cheat tools in a controlled environment to generate labeled “cheat” data (or collaboration with volunteers who can use existing cheats on a private server).^{[2][7]}

Academic projects have successfully built and trained cheat detectors using standard lab hardware, reinforcing that massive cloud resources are not necessary at this scale.^{[1][3][7]}

Estimated Timeframe

Given the constraints of a bachelor thesis (e.g., one semester or 4–6 months part-time), a realistic plan would:

- Month 1: Literature review, detailed scope definition, and initial server logging implementation.^{[11][3][^1]}
- Months 2–3: Data collection campaigns (legit and cheat sessions), data cleaning, and first feature engineering and baseline models.
- Month 4: Model refinement, server-side integration prototype, and experimental evaluation.
- Remaining time: Writing, visualization, and possible small extensions (e.g., comparison of classical ML vs a small neural network).

HaiX's team, with two developers and use of code-generating AI, produced a working anti-cheat prototype in approximately three months of focused effort, albeit with a budget of several thousand dollars. A single student will likely need more calendar time for comparable functionality, but a narrower, research-oriented scope (fewer cheat types, smaller user tests) is realistic within a standard thesis period.^[^2]

Constraints and Challenges

Data and Labeling Challenges

- Collecting enough labeled cheat data is difficult because ethical and legal rules require running cheats only on private test servers and with informed participants.^{[12][11]}
- Distinguishing high-skill human behavior from subtle cheats is a well-known challenge; many studies report trade-offs between catching more cheaters and avoiding false positives, which can severely damage player trust if innocent players are banned.^{[12][3]}
- Cheats evolve quickly and can adapt to known detection features, so models risk becoming outdated if they overfit to specific cheat implementations.^{[4][3]}

Technical and Practical Constraints

- Access to low-level telemetry depends on the game's server API; some desirable information (exact client-side view rendering, certain input details) may not be exposed server-side for CS2, limiting what can be observed.^{[4][1]}
- Real-time inference must be efficient to avoid adding latency or consuming too much CPU on shared servers; this constrains model size and complexity.^{[8][1]}
- Deployment in official matchmaking is impossible without cooperation from Valve or major platforms; the thesis must frame the system as a proof-of-concept for community servers or hypothetical integration.

Ethical and Legal Constraints

- Running or distributing cheats, even for research, can violate game terms of service; projects typically keep experiments confined to private servers and avoid releasing cheat code.^{[11][12]}
- Logging detailed player behavior raises privacy concerns, especially if data is linked to account identifiers or shared beyond the research group; many works address this by anonymizing data and following institutional review processes.^{[11][3]}

These constraints do not make the project impossible but require careful design, clear ethical boundaries, and very precise communication in the thesis.

Originality and Academic Value

AI-based anti-cheat is an active research area, with recent academic papers and industry products demonstrating both opportunities and open problems. The core idea of using machine learning or neural networks for cheat detection is therefore not novel by itself.^{[5][4][3][1]}

However, a bachelor thesis can still be original and valuable by:

- Targeting CS2 or a specific Counter-Strike environment with a concrete, reproducible dataset and code, which many commercial efforts do not publish.^{[1][2]}
- Focusing on a very specific behavior (e.g., server-side detection of trigger bots using minimal features such as per-tick fire events and crosshair positions) and thoroughly analyzing detection accuracy, latency, and false positives.^{[6][2]}
- Comparing simple classical models to small neural networks on the same features and discussing trade-offs between performance, interpretability, and resource usage in a server context.^{[3][7]}
- Exploring explainability, such as visualizing which parts of an aim trajectory caused the model to flag a player, inspired by work like XGuardian that emphasizes interpretable detection.^[^1]

Within the typical expectations of a bachelor thesis, combining a working prototype with a clear evaluation and literature-grounded discussion of limitations would be considered a solid and reasonably original contribution.

Overall Feasibility Assessment

Considering existing projects like HaiX's server-side anti-cheat, academic systems such as XGuardian, and multiple theses on ML-based cheat detection, a scoped-down AI-powered server-side anti-cheat for Counter-Strike is challenging but feasible as a bachelor thesis project. The main risk factors are data collection and labeling effort, engineering time for server integration, and managing ethical constraints.^{[7][3][2][1]}

If the project is scoped to one or two cheat types, uses manageable models, and emphasizes evaluation over polished production deployment, it aligns well with current research trends

and can produce both a technically interesting prototype and an academically valuable written thesis.^[6]^[3]^[^1]

References

1. XGuardian: Towards Explainable and Generalized AI Anti ... - Aim-assist cheats are the most prevalent and infamous form of cheating in First-Person Shooter (FPS)...
2. HaiX Reacts To Hate & Criticism About His AI ANTI-CHEAT - BUSINESS EMAIL ► KingHaiX7@yandex.ru
KICK ► <https://kick.com/kinghaix>
MAIN CHANNEL ► [https://www.you...](https://www.youtube.com/channel/UC...)
3. Detecting Aimbot Cheats in FPS Games Using Computer Vision - Aimbot cheats in first-person shooter (FPS) games automatically manipulate player input to target op...
4. Robust Vision-Based Cheat Detection in Competitive Gaming - Game publishers and anti-cheat companies have been unsuccessful in blocking cheating in online gaming...
5. Visual-Based Anti-Cheating Detection Network in FPS Games - IJETA
6. Anti-Cheat in FPS Game for Aimbot Detection
7. [PDF] Machine Learning to identify cheaters in online games - the result of this master thesis is the first step towards creating a reliable anti-cheat machine le...
8. AI-Powered Server Anti-Cheat for Unreal Engine: Hard Anti-Cheat - The pioneering AI-powered server anti-cheat solution for Unreal Engine, leveraging cutting-edge neur...
9. [PDF] Data Preparation and Analysis in Support to Cheating Detection
10. Comparative Study of Anti-cheat Methods in Video Games (CS 101) - This thesis will compare different anti-cheat methods, both on the client-side and server-side, and...
11. [PDF] Cheating and anti-cheating software in video games - OuluREPO - The aim of this thesis is to examine different types of cheating and anti-cheating software and the ...
12. Anti-Cheat and Cybersecurity in eSports and Gaming: A Case Study - This case study investigate the use of anti-cheat technology and cybersecurity implemented in gaming...